



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA
Escuela Profesional de Ingeniería de Sistemas

INFORME DE LABORATORIO
“Máquina Expendedora”

Curso: Sistemas de Información

Docente: Ing. Breno Luque Zuñiga

Peralta Sebán, Daniel Sebastián (2024016803)

Tacna – Perú
2026



INDICE

I.	INFORMACIÓN GENERAL.....	3
-	Objetivos.....	3
-	Equipos, materiales, programas y recursos utilizados.....	3
II.	MARCO TEORICO	3
III.	PROCEDIMIENTO	4
	Programa: Catálogo de Productos y Buscador:.....	4
	1. Diseño del Programa.....	4
	2. Tabla de Controles.....	4
	3. Código del Programa.....	6
IV.	ANALISIS E INTERPRETACION DE RESULTADOS	15
	CONCLUSIONES.....	15
	RECOMENDACIONES	15
	WEBGRAFIA.....	16



INFORME DE LABORATORIO

TEMA: Máquina Expendedora

I. INFORMACIÓN GENERAL

- **Objetivos:**
 - El desarrollo del sistema simulador de máquina expendedora.
 - La gestión de la lista de productos mediante listas dinámicas.
 - La aplicación de validaciones para controlar las compras.
 - La gestión del crédito y en su caso de cambio para el cliente.
 - La gestión del almacenamiento de las ventas en archivos de texto.
 - La gestión en cuanto a reportes de las ventas diarias, mensuales y anuales.
 - De los controles gráficos en Windows Forms de tipo dinámico.
 - El uso intensivo de programación orientada a objetos y manejo de archivos.
- **Equipos, materiales, programas y recursos utilizados:**
 - Computadora con sistema operativo Windows 8
 - Visual Studio
 - Lenguaje de programación C#.
 - .NET Framework

II. MARCO TEORICO

Las máquinas expendedoras automatizan procesos de venta permitiendo seleccionar productos, realizar pagos y entregar cambio automáticamente.

Programación Orientada a Objetos: Permite modelar entidades reales mediante clases y objetos. En este sistema, cada producto es representado por la clase Producto.

List: Estructura dinámica utilizada para almacenar los productos disponibles.

Manejo de archivos: El sistema utiliza archivos .txt para almacenar información de ventas de manera persistente.

LINQ: Se utiliza para buscar productos dentro de la lista mediante consultas eficientes.

DataGridView: Permite visualizar datos en forma tabular, facilitando la interacción con el usuario.



III. PROCEDIMIENTO

Programa: Catálogo de Productos y Buscador:

1. Diseño del Programa

2. Tabla de Controles

Control	Propiedad	Valor
Label1	Text	MÁQUINA EXPENDEDORA
	Font	bold type
Label2	Text	Reporte de ventas
	Font	bold type
Label3	Text	Código
Label4	Text	Credito
Label5	Name	lblCredito
	Text	Credito ingresado: S/. 0.00
Button1	Name	btn1
	Text	1
Button2	Name	btn2
	Text	2
Button3	Name	btn3
	Text	3
Button4	Name	btn4
	Text	4
Button5	Name	btn5



Button6	Text Name	5 btn6
Button7	Text Name	6 btn7
Button8	Text Name	7 btn8
Button9	Text Name	8 btn9
Button10	Text Name	9 btn0
Button11	Text Name	0 btnpunto
Button12	Text Name	. btnDel
Button13	Text Name	Eliminar btnCambio
Button14	Text Name	Cambio btnSalir
Button15	Text Name	Salir btnComprar
Button16	Text Name	Comprar btnDia
Button17	Text Name	Dia btnMes
Button18	Text Name	Mes btnAño
	Text	Año
Textbox1	Name	txtId
Textbox2	Name	txtNombre
Textbox3	Name	txtApellido
Textbox4	Name	txtTelefono
Textbox5	Name	txtDireccion
Textbox6	Name	txtBuscar
dataGridView1	Name	dgvReporte
flowLayoutPanel1	Name	flpProductos

3. Código del Programa

Funcionalidad 1: Declaración de variables principales

La lista almacena todos los productos disponibles en la máquina y la variable saldo almacena el crédito disponible del usuario.

```
public partial class Form1 : Form
{
    List<Producto> productos = new List<Producto>();

    decimal saldo = 0;
```

Define el archivo donde se almacenarán todas las ventas realizadas.

```
string archivoVentas = "ventas.txt";
```

Define cuántos productos aparecerán en cada fila de la interfaz.

```
readonly int[] filas = { 5, 5, 10, 10, 10, 10 };
```

Funcionalidad 2: Método Escribir

Permite ingresar números y símbolos en el cuadro de crédito simulando un teclado numérico.

```
11 referencias
void Escribir(string valor)
{
    txtCredito.Text += valor;
}
```

Funcionalidad 3: Clase Producto

Representa cada producto disponible dentro de la máquina expendedora.

Propiedades:

- Código
- Precio

```
5 referencias
public class Producto
{
    5 referencias
    public string Codigo { get; set; }
    6 referencias
    public decimal Precio { get; set; }
}
```

Funcionalidad 4: Inicialización del sistema

Evento Form1_Load: Se configura completamente la interfaz al iniciar la aplicación.

Limpieza del crédito

```
1 referencia
private void Form1_Load(object sender, EventArgs e)
{
    txtCredito.Clear();
}
```

Configuración del reporte

```
dgvReporte.ColumnCount = 4;
```

Columnas creadas

```
dgvReporte.Columns[0].Name = "Fecha";  
dgvReporte.Columns[1].Name = "Hora";  
dgvReporte.Columns[2].Name = "Código";  
dgvReporte.Columns[3].Name = "Precio";
```

Evitar filas vacías

```
dgvReporte.AllowUserToAddRows = false;
```

Carga de productos y Mostrar productos

```
    CrearProductos();  
  
    MostrarProductos();  
}
```

Funcionalidad 5: Creación automática de productos

Método CrearProductos: El sistema genera automáticamente productos con códigos y precios distintos.

```
1 referencia  
void CrearProductos()  
{  
    int contador = 1;  
  
    Random rnd = new Random();  
  
    foreach (var cantidad in filas)  
    {  
        for (int i = 0; i < cantidad; i++)  
        {  
            productos.Add(new Producto  
            {  
                Codigo = contador.ToString("000"),  
  
                Precio = Math.Round(  
                    (decimal)(rnd.NextDouble() * 5 + 1), 2  
                ));  
  
            contador++;  
        }  
    }  
}
```

Funcionalidad 6: Visualización dinámica de productos

Método MostrarProductos: Los productos son creados visualmente de manera dinámica, simulando compartimentos de una máquina expendedora real. Limpiando la interfaz y creaciones de paneles. Muestra código y precio



```
1 referencia
void MostrarProductos()
{
    flpProductos.Controls.Clear();

    int index = 0;

    int y = 10;

    foreach (var cantidad in filas)
    {
        int x = 10;

        for (int i = 0; i < cantidad; i++)
        {
            var p = productos[index];

            Panel item = new Panel
            {
                Width = 70,
                Height = 60,
                Left = x,
                Top = y,
                BorderStyle = BorderStyle.FixedSingle
            };

            Label lblCod = new Label
            {
                Text = p.Codigo,

                ForeColor = Color.DarkOrange,

                Font = new Font(
                    "Arial",
                    10,
                    FontStyle.Bold),

                Top = 5,
                Left = 15,

                AutoSize = true
            };

            Label lblPrecio = new Label
            {
                Text = "S/ " + p.Precio.ToString("0.00"),

                Top = 30,
                Left = 8,

                AutoSize = true
            };

            item.Controls.Add(lblCod);

            item.Controls.Add(lblPrecio);

            flpProductos.Controls.Add(item);

            x += 75;

            index++;
        }

        y += 70;
    }
}
```


MÁQUINA EXPENDEDORA

001 S/ 2.35	002 S/ 1.58	003 S/ 3.59	004 S/ 5.76	005 S/ 3.11					
006 S/ 4.03	007 S/ 5.45	008 S/ 3.27	009 S/ 1.22	010 S/ 4.19					
011 S/ 2.98	012 S/ 1.27	013 S/ 5.84	014 S/ 2.62	015 S/ 3.26	016 S/ 3.98	017 S/ 4.98	018 S/ 3.03	019 S/ 4.19	020 S/ 4.19
021 S/ 1.76	022 S/ 2.56	023 S/ 4.01	024 S/ 5.24	025 S/ 2.10	026 S/ 2.25	027 S/ 4.07	028 S/ 4.70	029 S/ 4.70	030 S/ 4.70
031 S/ 1.01	032 S/ 1.84	033 S/ 3.75	034 S/ 4.73	035 S/ 4.17	036 S/ 5.51	037 S/ 4.28	038 S/ 4.92	039 S/ 4.92	040 S/ 4.92

Funcionalidad 7: Botones numéricos

Cada botón agrega un número al crédito ingresado por el usuario.

```
1 referencia
private void btn0_Click(object s, EventArgs e) => Escribir("0");
1 referencia
private void btn1_Click(object s, EventArgs e) => Escribir("1");
1 referencia
private void btn2_Click(object s, EventArgs e) => Escribir("2");
1 referencia
private void btn3_Click(object s, EventArgs e) => Escribir("3");
1 referencia
private void btn4_Click(object s, EventArgs e) => Escribir("4");
1 referencia
private void btn5_Click(object s, EventArgs e) => Escribir("5");
1 referencia
private void btn6_Click(object s, EventArgs e) => Escribir("6");
1 referencia
private void btn7_Click(object s, EventArgs e) => Escribir("7");
1 referencia
private void btn8_Click(object s, EventArgs e) => Escribir("8");
1 referencia
private void btn9_Click(object s, EventArgs e) => Escribir("9");
1 referencia
private void btnpunto_Click(object s, EventArgs e) => Escribir(".");
```

Funcionalidad 8: Compra de productos

Evento btnComprar_Click:

La lógica de compra controla:

- Existencia del producto.
- Crédito suficiente.
- Descuento del saldo.
- Registro de venta.



```
1 referencia
private void btnComprar_Click(object sender, EventArgs e)
{
    string codigo = txtCodigo.Text.Trim();

    Producto producto =
        productos.FirstOrDefault(x => x.Codigo == codigo);

    if (producto == null)
    {
        MessageBox.Show("Código no encontrado");
        return;
    }

    decimal ingresado = 0;

    if (txtCredito.Text != "")
    {
        decimal.TryParse(txtCredito.Text, out ingresado);
    }

    decimal total = saldo + ingresado;

    if (total < producto.Precio)
    {
        MessageBox.Show("Crédito insuficiente");
        return;
    }

    saldo = total - producto.Precio;

    MessageBox.Show(
        "Compra realizada\n\n" +
        "Código: " + producto.Codigo +
        "\nPrecio: S/ " +
        producto.Precio.ToString("0.00") +
        "\nSaldo restante: S/ " +
        saldo.ToString("0.00"));

    GuardarVenta(producto);

    txtCredito.Clear();

    txtCodigo.Clear();

    lblCredito.Text =
        "Crédito disponible: S/ " +
        saldo.ToString("0.00");
}
```

Código
4654

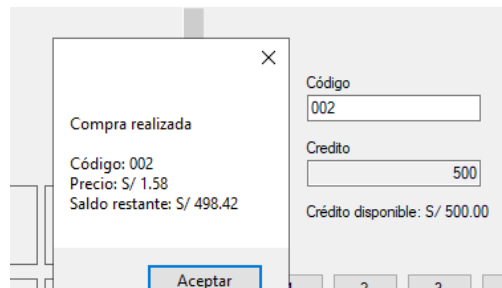
Credito

Credito ingresado: S/. 0.00

1	2	3
4	5	6
7	8	9
.	0	

Código no encontrado

Aceptar



Funcionalidad 9: Registro de ventas

Método GuardarVenta: Cada compra queda registrada permanentemente en un archivo de texto.

Datos guardados:

- Fecha
- Hora
- Código
- Precio

```
1 referencia
void GuardarVenta(Producto p)
{
    string linea =
        DateTime.Now.ToString("yyyy-MM-dd") + "|" +
        DateTime.Now.ToString("HH:mm:ss") + "|" +
        p.Codigo + "|" +
        p.Precio;

    File.AppendAllText(archivoVentas,
        linea + Environment.NewLine);
}
```

Funcionalidad 10: Mostrar reportes

Método MostrarReporte: El sistema genera reportes dinámicos según el periodo seleccionado.

Verifica si el archivo "ventas.txt" y si existe se lee las líneas

```
3 referencias
void MostrarReporte(string tipo)
{
    dgvReporte.Rows.Clear();

    if (!File.Exists(archivoVentas))
    {
        return;
    }

    string[] lineas = File.ReadAllLines(archivoVentas);
```

Separa los datos

```
foreach (string linea in lineas)
{
    string[] datos = linea.Split('|');

    DateTime fecha = DateTime.Parse(datos[0]);

    bool mostrar = false;
```

Filtro por día

```
if (tipo == "DIA")
{
    mostrar = fecha.Date ==
        DateTime.Now.Date;
}
```

Filtro por mes



```
if (tipo == "MES")
{
    mostrar = fecha.Month ==
        DateTime.Now.Month;

    mostrar &= fecha.Year ==
        DateTime.Now.Year;
}
```

Filtro por año

```
if (tipo == "AÑO")
{
    mostrar = fecha.Year ==
        DateTime.Now.Year;
}
```

Muestra los datos

```
if (mostrar)
{
    dgvReporte.Rows.Add(
        datos[0],
        datos[1],
        datos[2],
        "S/ " + datos[3]);
}
```

Reporte de ventas				
	Fecha	Hora	Código	Precio
▶	2026-05-12	21:14:01	001	S/ 1.96
	2026-05-12	21:14:36	006	S/ 3.91
	2026-05-12	21:27:58	001	S/ 3.56
	2026-05-12	21:28:15	006	S/ 2.41
	2026-05-12	21:28:28	002	S/ 3.88
	2026-05-12	21:34:11	005	S/ 5.88

Día

Mes

Año

Funcionalidad 11: Botones de reporte

Cada botón ejecuta un filtro distinto sobre las ventas registradas.

```
1 referencia
private void btnDia_Click(object sender, EventArgs e)
{
    MostrarReporte("DIA");
}

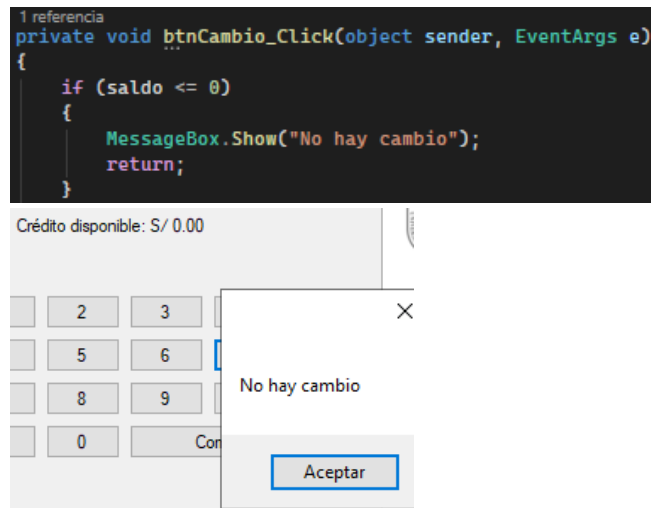
1 referencia
private void btnMes_Click(object sender, EventArgs e)
{
    MostrarReporte("MES");
}

1 referencia
private void btnAño_Click(object sender, EventArgs e)
{
    MostrarReporte("AÑO");
}
```

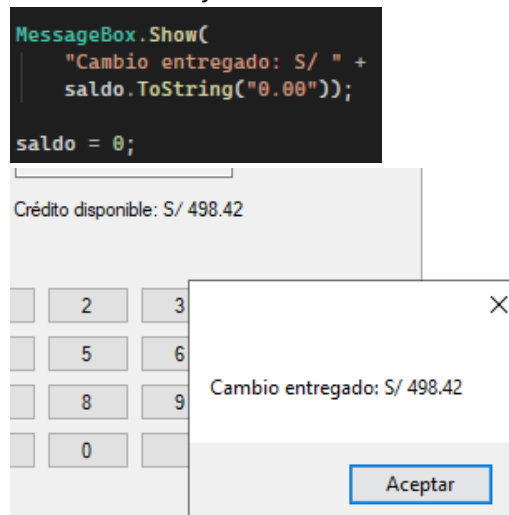
Funcionalidad 12: Entrega de cambio

Evento btnCambio_Click: Permite devolver el crédito restante al usuario.

Validación de saldo



Mostrar cambio y reinicio del saldo



Actualización visual

```
lblCredito.Text =
    "Crédito disponible: S/ 0.00";
txtCredito.Clear();
}
```

Funcionalidad 13: Salida del sistema y limpiar crédito

Evento `btnSalir_Click`: Cierra completamente la aplicación.

```
1 referencia
private void btnSalir_Click(object sender, EventArgs e)
{
    Application.Exit();
}
```

Evento `btnDel_Click`: Permite borrar el monto ingresado.

```
1 referencia
private void btnDel_Click(object sender, EventArgs e)
{
    txtCredito.Clear();
}
```

Funcionalidad 14: Actualización automática del crédito

Evento txtCredito_TextChanged: El crédito disponible se actualiza automáticamente mientras el usuario escribe.

```
1 referencia
private void txtCredito_TextChanged(object sender, EventArgs e)
{
    decimal.TryParse(txtCredito.Text, out decimal ingresado);

    decimal total = saldo + ingresado;

    lblCredito.Text =
        "Crédito disponible: S/ " +
        total.ToString("0.00");
}
```



IV. ANALISIS E INTERPRETACION DE RESULTADOS

¿Qué indican los resultados?

- El sistema simula correctamente el funcionamiento de una máquina expendedora
- Compras y reportes funcionan adecuadamente.
- Crédito y cambio se calculan correctamente.

¿Que se ha encontrado?

- Este sistema permite gestionar las ventas de forma permanente.
- La implementación de los controles permite construir de forma más visual los productos.
- LINQ permite gestionar los productos de forma sencilla.
- La lógica de validación permite evitar errores en la compra.

CONCLUSIONES

- Se creó una máquina expendedora totalmente funcional utilizando C# y Windows Forms; el sistema permite gestionar los productos, los créditos, las compras y los reportes de forma eficiente.
- Se han implementado los controles dinámicos de forma que se logre una interfaz más interactiva.
- El manejo de archivos permite almacenar las ventas de forma permanente.
- Los reportes sirven para la visualización de cierta información sobre las ventas realizadas.
- La programación orientada a objetos mejora la organización y el mantenimiento del sistema.
- Este proyecto es una buena base para sistemas más robustos de venta e integración.
- La lógica de validación permite el correcto funcionamiento de las compras en el proceso de compra.
- Los reportes permiten ver información importante sobre las ventas realizadas.
- La programación orientada a objetos permite una mejor organización y mantenimiento del sistema.

RECOMENDACIONES

- Implementar control de stock real.
- Incorporar base de datos.



WEBGRAFIA

BillWagner. (s. f.). Operadores y expresiones: enumerar todos los operadores y expresiones - C# reference. Microsoft Learn. Recuperado de <https://learn.microsoft.com/es-es/dotnet/csharp/language-reference/operators/>

Arreglos en C#. (2014). C# Paso a Paso. Recuperado de <https://csharp-facilito.blogspot.com/2013/07/arreglos-en-c-sharp.html>

Turrado, J. (2022). Introducción rápida a LINQ con C#: manejar información en memoria nunca fue tan sencillo - campusMVP.es. campusMVP.es. Recuperado de https://www.campusmvp.es/recursos/post/introduccion-rapida-a-linq-con-c-sharp.aspx?srsId=AfmBOoqx5_WZg2mtHdV0_YFJiBZsZ5dj_t3Us87MablaeW1k-0upk2xK